30 分钟那道线

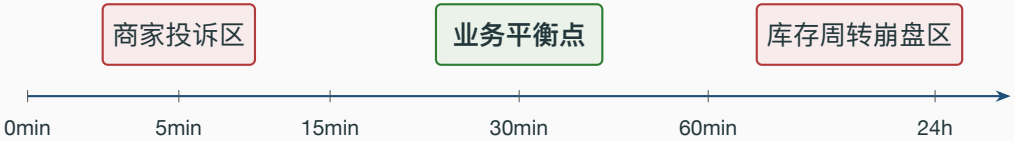
关单时机的业务博弈与双保险机制 (gomall 业务侧 deck)

gomall · 业务侧 deck

今日大纲

30 分钟是怎么定的

这道线两头都得罪人



- 商家侧抱怨：”我刚下完单 5 分钟就被取消”——5 分钟切支付方式 / 等验证码都不够。
- 运营侧抱怨：”双 11 锁单 24 小时谁也抢不到”——爆款被恶意囤一整天，正经买家流失。
- 30 分钟是**双方都能接受的最大公约数**，不是技术参数，是**业务的合同条款**。

*repository/rabbitmq/order_delay.go:22 OrderCancelDelay = 30 * time.Minute*

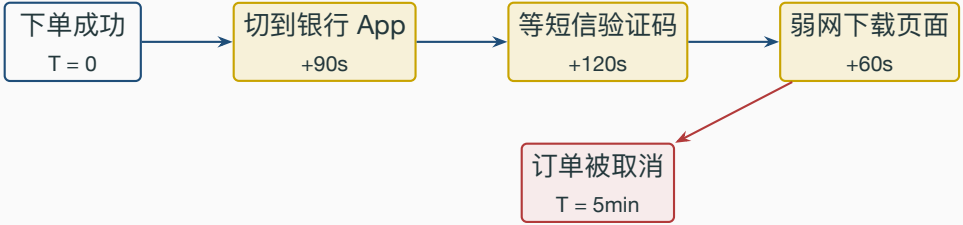
为什么不是 5min / 15min / 60min

30min 是工业界经验值（京东 30min / 天猫 30min–1h / 拼多多 15min）。改这个值要谈 **5 个角色**：商家 / 运营 / 风控 / 客服 / 财务——代码侧只落到 `OrderCancelDelay` 一行。

repository/rabbitmq/order_delay.go:21–22 30min 常量

短 / 长的代价：双 11 的两个真实场景

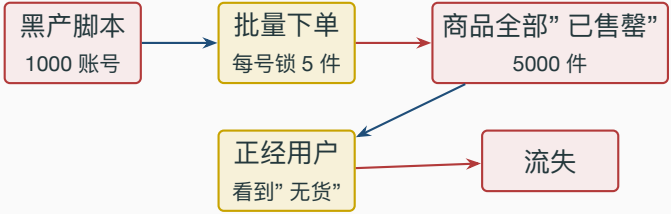
场景 A：5 分钟太短 = 付款流失



- 5 分钟里要做：切银行 / 等短信 / 弱网下载 / 输密码 / 人脸识别。
- 客单价越高，付款链路越长（双卡 / 分期 / 信用卡确认）。
- **真实数字**：电商支付链路 p95 = 120s，p99 = 240s。5min 砍 p99 用户。

支付链路时延来自业界 *RUM* 调研，非 *gomall* 实测

场景 B：24 小时太长 = 恶意囤货



- 24 小时锁单 = 黑产**零成本**囤光商品（不付款，第二天自动取消）。
- 商家库存看上去” 卖完了”，运营找不到原因。
- 真实用户进来看到” 已售罄”→ 流失。GMV 没增长，库存周转崩盘。
- 这是**库存被武器化**：把” 自动关单延迟” 变成攻击工具。

deck 11 inventory-business 第 *4* 节恶意锁单同主题

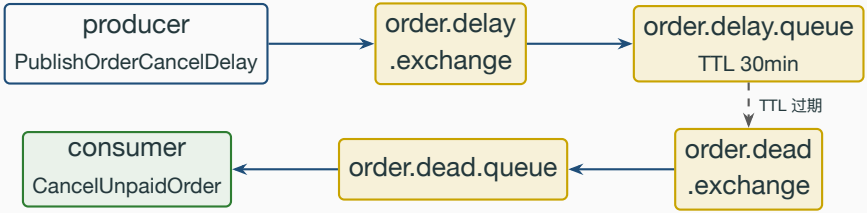
RMQ + Cron 双保险：单链路都不够

为什么不能只留一条链路

- **只 RMQ**：进程挂 / 丢消息 / nack 进 DLX = 订单永远停 UnPaid。
- **只 Cron**：5min 间隔 + 单批 100 上限，库存释放 lag 偏大。
- **双保险**：RMQ 30min 准点 + Cron 15min 窗口 / 5min 节奏。
- **Cron 用 15min 而非 30min**：RMQ 正常时订单 30min 已被关掉、Cron 不抢功；RMQ 挂时 Cron 立刻接管，最多多锁 15–20min。

order_delay.go:24–56；order_task.go:14–48,:16 GetTimeoutOrders(15,100)

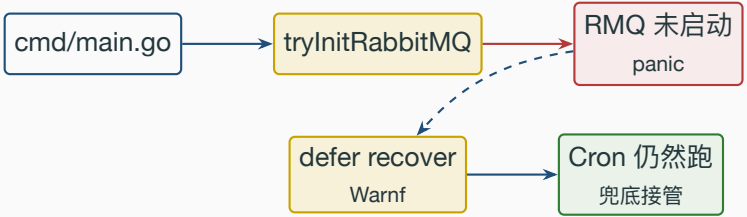
RMQ 延迟队列拓扑：死信交换换出” 延迟”



- RabbitMQ 没有原生延迟队列——**TTL + 死信交换**是工业界标准做法。
- 消息进 delay 队列后 30min ” 未被消费”→ TTL 触发 → 流向 dead 交换 → 真正的消费者吃。
- 持久化（`DeliveryMode: Persistent`）+ 手动 ack，避免消费者重启丢消息。

repository/rabbitmq/order_delay.go:24–56 拓扑；:58–72 publish；:74–105 consume

RMQ 启动失败时的静默降级 = 双保险的最佳实证

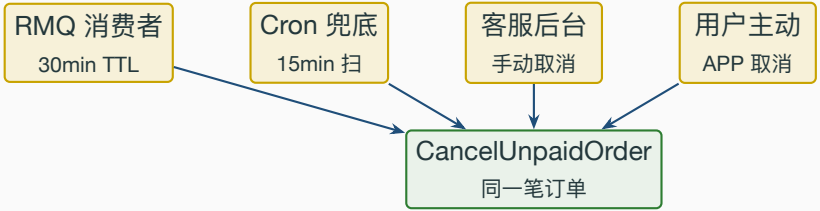


- gomall 压测报告里写得清楚：**"RabbitMQ 未启动 (不影响 HTTP 链路)"**。
- RMQ 启动失败 = recover 捕获 panic + 打 Warn 日志，HTTP 服务继续接客。
- 关单链路降级为**只走 Cron**，业务承诺打折但不崩。
- 这是**双保险设计的最佳实证**：缺一不可，但缺一不死。

cmd/main.go:49–58 tryInitRabbitMQ 静默降级；stressTest/REPORT.md 第 7 行

CancelUnpaidOrder 必须幂等

至少 4 个调用方会触发同一笔关单



- 同一笔订单可能**同时**被 4 个来源触发关单（高并发下并不罕见）。
- 例：用户付款失败手动取消 → Cron 也扫到 → RMQ 也到期 → 三次调用打到 `CancelUnpaidOrder`。
- 如果不幂等：库存释放 3 次（实际欠库）、outbox 写 3 条 `order.cancelled`（下游处理 3 次）。

service/order_cancel.go:19–63 单一入口接 *4* 个调用方

CloseOrderWithCheck：用 WHERE 条件兜住幂等

```go
185 func (dao *OrderDao) CloseOrderWithCheck(orderNum uint64) (bool, error) {
186     res := dao.DB.Model(&model.Order{}).
187         Where("order_num=? and type=?", orderNum, consts.UnPaid).
188         Update("type", consts.Cancelled)
189     if res.Error != nil { return false, res.Error }
190     return res.RowsAffected > 0, nil
191 }
```

Listing 1: repository/db/dao/order.go:185–196

- `WHERE type=UnPaid`：只有 UnPaid 会被改成 Cancelled，已 Cancelled / Paid 直接 RowsAffected=0。
- 第二次调用 RowsAffected=0 → closed=false → 直接 return，不释放库存 / 不写 outbox。

order.go:185–196 幂等 SQL；order_cancel.go:30–57 closed 旗标

幂等失守会发生什么

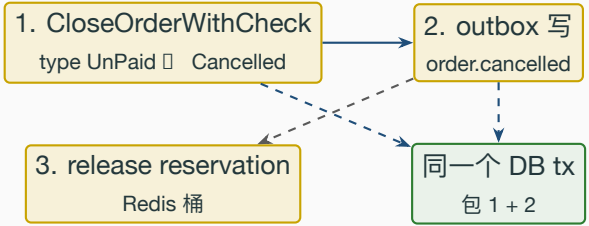
失守场景	用户侧后果	业务侧后果
库存释放 N 次	看到”虚高”库存	实际欠库，超卖必赔
outbox 发 N 条 cancelled	下游收 N 次	ES 删 N 次、商家通知 N 次
状态机回退 (Paid → Cancelled)	钱付了订单消失	资损 P0

- 最严重的是**状态机回退**：用户已经付了款的订单被误关单 = 资损。
- gomall 用 `type=UnPaid` 这个 WHERE 死锁住——已付款的订单永远不会被 cancel 路径碰到。
- 这条 SQL 是关单链路的**最后一道防线**，绝对不能改。

repository/db/dao/order.go:185–196；consts/order.go UnPaid / Paid / Cancelled 状态码

关单是金融事件，不是状态变更

一次关单 = 三个动作必须原子发生



- **1 + 2 必须在同一个 DB 事务**：状态变更和事件发布原子绑定（outbox 模式）。
- **3 在 tx 之外**：Redis 不支持 2PC，failed → 留 reserved 给下次对账修正，胜过参与分布式事务。
- 这是**关单作为金融事件**的最低要求：状态 + 事件 + 库存三件套必须协调。

service/order_cancel.go:31–51 DB tx 包 1+2；:59–61 tx 外释放 Redis

不能给商家发”发货通知”

- `order.cancelled` 下游绝对**不能**包含”通知商家发货”——商家收到”发货” → 真发货 → 用户没付款 → 商家亏运费 + 货损。
- 合理下游：ES 删索引 / 库存对账 / 客服工单关闭 / 推送”订单已取消”。
- **关键不变量**：`order.cancelled` 永远只对应 type=UnPaid（已 Paid 走 `order.refunded`）。

> 关单是触发一组下游业务动作的金融事件——谁该消费、谁不该，事件设计时定死。

对账兜底：凌晨低峰跑 `SUM(reserved)` vs `SUM(UnPaid.Num)` 偏差告警，是双保险都漏时的最后一道闸（运维 backlog 第 1 项，待落地）。*service/order_cancel.go:40–50 outbox payload；repository/cache/inventory.go:127–138* 对账入口

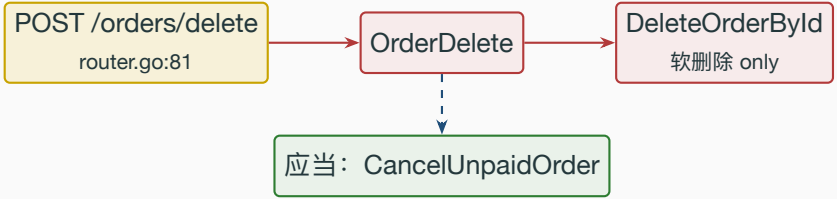
遗留与改进

遗留 1：三处时长口径不一致

- 三处口径**业务含义不同**（关单时机 / 兜底窗口 / 内部排序），但都涉及”15min 这个数”。
- 风险：未来改 30min → 15min 时，是否要同步改 Cron / ZSet?
- 改进：抽到 `consts/order.go` 单独命名（`OrderCloseTTL` / `OrderCronBacklog` / `OrderZSetOffset`）。

repository/rabbitmq/order_delay.go:22；service/order_task.go:16；service/order.go:88–92

遗留 2：`orders/delete` 绕过 CancelUnpaidOrder



- `routes/router.go:81` 的 `orders/delete` 直接调 OrderDelete → DeleteOrderById（GORM 软删除）。
- **漏了 release reservation 和 outbox 写**——Redis 桶仍占库存，下游 ES 不知道订单消失。
- 建议要么对 UnPaid 改走 CancelUnpaidOrder，要么禁掉这个 endpoint。

routes/router.go:81；service/order.go:168–192；repository/db/dao/order.go:149–154

遗留 3：Cron 表达式 bug——意图 5min 一次, 实际 60 次

```go
14 c := cron.New(cron.WithSeconds())
15 orderService := new(service.OrderTaskService)
16 _, err := c.AddFunc("* */5 * * * *", func() { /* ... */ })
```

Listing 2: initialize/cron.go:14–16 触发表达式

- 6 字段 cron 秒分时日月周：`* */5 * * * *` = 第 0/5/10/…分钟里**每秒都触发**, 5min 窗口内打 60 次。
- 业务意图” 每 5 分钟跑一次”，正确写法 `0 */5 * * * *`（秒位 = 0）。
- 影响：DB 被无谓打 60 倍 + 日志被淹没。功能层面靠 CloseOrderWithCheck 幂等没产生 bug。

initialize/cron.go:16；robfig/cron/v3 with-seconds 字段顺序

遗留 4：RMQ 长时间不通 / Cron 长时间不跑都没告警

- RMQ 静默降级**很优雅**（cmd/main.go:49–58 recover），但运维侧**看不到**——只有一行 Warn 日志，没有 metric。
- Cron 进程如果死掉（panic 没 recover / 进程被 OOM kill），**没有外部心跳监控**知道。
- **改进**：
  - RMQ 健康度上 Prometheus（`rabbitmq_up` gauge）+ 告警规则。
  - Cron 任务每次跑完写 `last_run_ts` 到 Redis，超过 10min 没更新就告警。
  - DLX 队列堆积 > N 条告警（消费者 nack 进 DLX = 真有 bug）。

cmd/main.go:49–58；initialize/cron.go:14–28 当前无心跳；repository/rabbitmq/order_delay.go:98 nack requeue 后进 DLX

业务承诺总表

业务承诺总表 + 压测复核

承诺	实现	复核	
30min 准点关单	RMQ TTL 30min	RMQ 单测 + 端到端	六条承
兜底关单	Cron 15min 窗口 / 5min 节奏	RMQ 关停后 Cron 接管	
幂等关单	SQL WHERE type=UnPaid	4 并发 → 1 单 cancel	
库存原子释放	DB tx 内 close + outbox	Redis tx 外异步 release	
已付款绝不误关	WHERE 死锁 type=UnPaid	状态机不可回退	
下游事件单次	outbox 在 tx 内写一次	cancelled 事件唯一	

诺缺一不可。压测复核：`orders/create` 单机 50,319 RPS / p95 2.33ms（远低于 P0 SLO 500ms）；幂等中间件 50 VU 15s 重放 755,033 次仍只 1 单；**RabbitMQ 未启动仍 0 错**——cmd/main.go:49–58 静默降级间接证明双保险的可降级性。*repository/rabbitmq/order_delay.go + service/order_cancel.go + service/order_task.go 三件套；stressTest/REPORT.md § 链路压测；cmd/main.go:49–58*

回到那条 30 分钟的线

- 30 分钟不是延迟队列的 TTL 参数，是**商家、运营、风控、客服、财务**五方的合同。
- RMQ + Cron 不是技术冗余，是**业务承诺**：哪怕 RMQ 整个挂了，库存最多多锁 35min。
- CancelUnpaidOrder 幂等不是” 代码优雅”，是因为**真的有 4 个调用方**并发触发。
- 关单是金融事件——下游消费者必须设计成**不能给商家发货通知**。
- 三处遗留（口径 / orders/delete / cron 表达式）说明业务承诺到代码的最后一公里需要持续打磨。

关单治理的核心是**把”30 分钟” 落到 6 条承诺 + 双保险 + 幂等 SQL 上**。

业务 Q&A（上）

Q1：为什么 30min，不能改成 1 小时?

A：60min 的代价是双 11 锁单太久 / 爆款一直显示” 已售罄” / 库存周转崩盘。30min 是商家投诉曲线和库存周转的工业界经验值。改要 5 个业务角色一起谈。

Q2：RMQ 挂了多久能发现?

A：当前**没有外部告警**——cmd/main.go:49–58 只是 Warn 日志。改进项第 4 条。短期靠 Cron 兜底（15–20min 内库存释放）。

Q3：用户 APP 主动取消 + RMQ 同时到期会出错吗?

A：不会。CloseOrderWithCheck 用 `WHERE type=UnPaid` 兜底——第一个成功的 RowsAffected=1，第二个 RowsAffected=0 直接 return。

Q4：客服后台关单和这条链路是同一个吗?

A：业务侧应当统一走 `CancelUnpaidOrder`。如果客服有” 强制关单” 需求（用户付了但要退），那是 `order.refunded` 链路，不在本 deck 范围内。

业务 Q&A（下）

Q5：Cron 表达式 bug 为什么没人发现？

A：幂等关单 SQL 兜住了功能正确性，但 DB QPS 被无谓放大 60 倍；监控里能看出 SELECT type=UnPaid 异常高频。改 `0 */5 * * * *` 即可。

Q6：为什么不直接用 Redis ZSet 替代 RMQ？

A：order.go:88–92 的 ZSet 只用作内部状态。换 ZSet 替代 RMQ 要自己写消费循环 + ack + 重试 + DLX——RMQ 已经打包好了，当前业务量性价比更高。

Q7：对账脚本什么时候上？

A：运维 backlog 第 1 项。当前靠商家反馈” 库存对不上” 才查 GetStockSnapshot，不是好状态。

Q8：双保险能保证 100% 不漏单吗？

A：不能。outbox 提交但 publisher 挂掉这条漏，仍要靠**对账**做最终兜底。

代码位置一览

30min 常量 *repository/rabbitmq/order_delay.go:21–22*

RMQ 延迟拓扑 *repository/rabbitmq/order_delay.go:24–56*

RMQ publish *repository/rabbitmq/order_delay.go:58–72*

RMQ consume *repository/rabbitmq/order_delay.go:74–105*

CancelUnpaidOrder *service/order_cancel.go:19–63*

CloseOrderWithCheck 幂等 SQL *repository/db/dao/order.go:185–196*

Cron 任务 *service/order_task.go:14–48*

Cron 注册 (含表达式 bug) *initialize/cron.go:13–28*

RMQ 启动失败静默降级 *cmd/main.go:49–58*

下单时发 RMQ + ZSet *service/order.go:88–98*

orders/delete 绕过关单链路 *routes/router.go:81；service/order.go:168–192*

Cron 兜底窗口 15min *service/order_task.go:16；repository/db/dao/order.go:162–170*